Week 2 / 6

Git 的基本架構

Git Architecture

GitHub for UI Designers

AGENDA

這週會講什麼

What we'll cover today

01	**Git vs GitHub** The first thing people confuse	最常搞混的第一題 GitHub hosts Git repos + a collaboration layer
02	**從 GitHub Clone 到電腦** Cloning a repo onto your machine	新人上路的第一步 Getting started — the first step
03	**三個空間 + 本機 vs 遠端** Three zones + Local vs Remote	程式碼的旅程 The journey of your code
04	**同一份檔案的三種狀態** Three states of one file	為什麼同事看不到我的改動？ Why colleagues don't see your edits
05	**Branch — 平行世界的工作空間** Branch — parallel workspaces	Git 最強大的功能之一 One of Git's most powerful features
06	**push 被擋了，怎麼辦** When commit & push gets rejected	安全的處理方式 Safe way to handling the rejections

COMPARISON

Git or GitHub？

GitHub hosts Git repos and adds a collaboration layer on top

	Git	GitHub
是什麼 What	軟體 / 指令（CLI） Software / CLI tool	網站 / 服務 Website / service
在哪裡 Where	你的電腦 On your computer	github.com（瀏覽器） github.com (browser)
怎麼用 How	打指令 Type commands	點按鈕、看頁面 Click, view pages

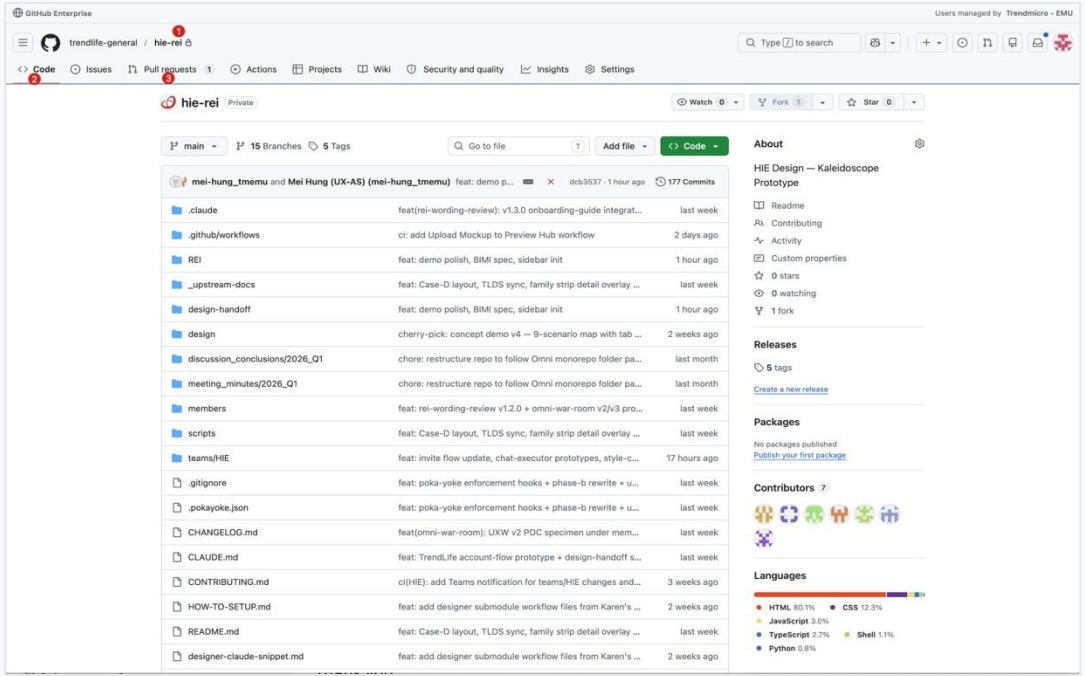
Git 本身就能跑在任何伺服器；GitHub 在 Git 之上加了 Pull Request、Review、Issues 等協作功能

Git can run on any server. GitHub adds collaboration features (PRs, reviews, issues) on top.

GITHUB PAGE

GitHub 重點頁面導覽

Three things to spot on a repo page



1 **Repo 名稱**

Repository name

trendlife-general / hie-rei = 帳號 / 專案名。Repo（repository）就是一個 Git 專案的容器。

owner / repo. A repository is the container for one Git project.

2 **Code**

Code tab

看檔案、取得 clone 網址、下載 zip。預設停在這頁。

Browse files, get the clone URL, download zip. Default landing tab.

3 **Pull Requests**

Pull requests

請別人 review 你的改動、看別人的改動。設計師最常進的頁面。

Request a review on your changes, or review others'. Designers' most-used tab.

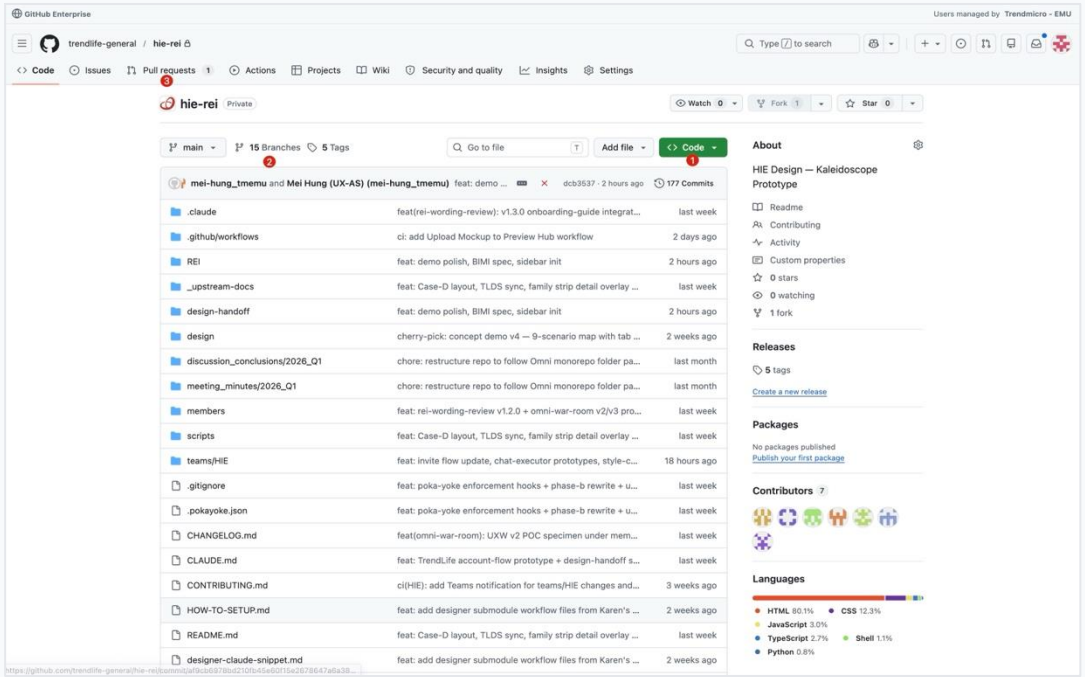
設計師最重要的兩個：Code（取得連結）與 Pull Requests（請人 review）

Two tabs that matter most: Code (get the URL) and Pull Requests (ask for review)

COMMON ACTIONS

常用操作清單

Where to click for the things you'll do most



1

Code 按鈕

Code button

右上方綠色按鈕。點開複製 HTTPS 網址 → 貼給 Claude Code 就能 clone 到本機。

Top-right green button. Copy the HTTPS URL → give it to Claude Code to clone.

2

Branches

Branches list

看現在有哪些 branch、誰在做什麼。也能從這裡刪掉舊 branch。

See active branches and who's working on what. Delete old ones from here.

3

Pull requests

Pull requests tab

建立新的 PR、看別人開的 PR、留 review 留言。Branch 改完要被 merge 都從這裡走。

Open a new PR, review others', leave comments. The merge gate for every branch.

90% 的設計師日常，這三個入口就夠用：拿程式碼、切換 branch、發 / 看 PR

90% of designer-daily flows live in these three: get code, switch branch, open / review a PR.

CLONE

新人上路：把專案 Clone 到電腦

Getting started — cloning a project

你只需要準備兩件事

Only two things to prepare

1 Repo 的 Clone 網址

The clone URL

從 GitHub 主頁右上角的綠色 Code 按鈕取得

From the green Code button at the top-right of the repo page

工程師會提供 repo 連結 · Engineer provides it

2 本機資料夾的位置

Where on your machine

你自己決定要放在哪個資料夾

You decide which local folder to use

建議 ~/work/ 底下分層管理 · Recommended: ~/work/ tree

準備好 → 告訴 Claude Code → Agent 完成剩下的步驟 · Ready → tell Claude Code → agent handles the rest

CLONE

從 GitHub 取得 Clone 網址

Two clicks to copy the HTTPS link

1

找到右上方的綠色 Code 按鈕，點擊

Find the green Code button at the top-right, click it

2

選 HTTPS 標籤，點右邊的複製圖示

Select the HTTPS tab, then click the copy icon on the right

HTTPS vs SSH

初次使用選 HTTPS

First time? Pick HTTPS

CLONE

決定本機資料夾位置

Pick a folder on your machine

```
~/work/
├── company/        ← company repos
├── personal/       ← personal
└── learning/       ← practice
```

選擇原則

Principles

- **集中管理** — Centralized
  全部放同一個父資料夾 · One parent folder
- **路徑簡短** — Keep paths short
  避免太深的巢狀 · Avoid deep nesting
- **英文路徑** — English only
  不要用中文或空格 · No CJK or spaces

放哪裡其實不嚴格 — 重點是路徑乾淨、找得到、自己順手

Where doesn't matter much — clean path, easy to find, comfortable for you.

CLONE

Clone 失敗？三個最常見的原因

Three common clone errors

`Permission denied`

原因 Cause	還不是這個 repo 的成員 Not a member of this repo yet	解法 Fix	請工程師或主管把你加入 repo Ask an engineer / lead to add you

`Repository not found`

原因 Cause	網址錯誤或 private repo 未登入 Wrong URL, or private repo not logged in	解法 Fix	確認網址、確認 GitHub 帳號已登入 Verify URL & that you're signed in

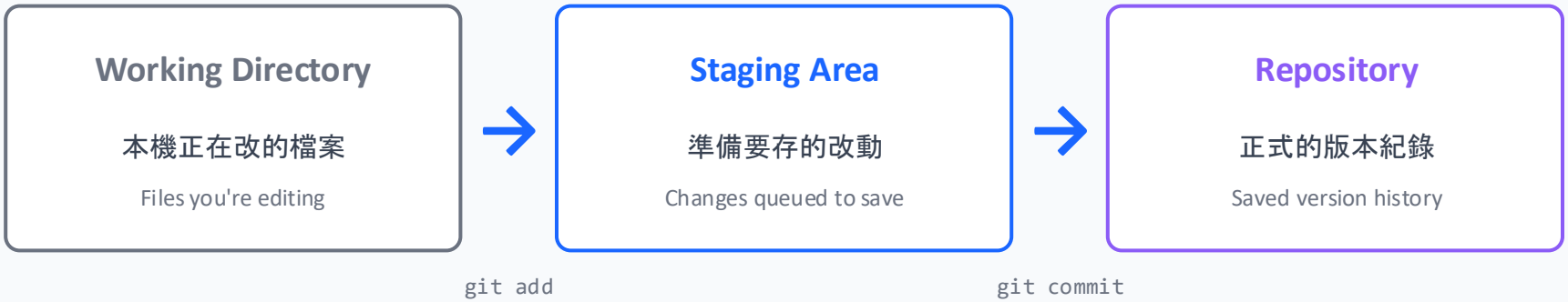
`destination already exists`

原因 Cause	目標位置已有同名資料夾 A folder with that name already exists	解法 Fix	換個路徑，或請 Claude Code 看看那個資料夾 Pick a new path, or ask Claude Code to check

ARCHITECTURE

三個空間：程式碼的旅程

Working Directory → Staging → Repository



為什麼需要 Staging？

Why staging?

你可能改了 10 個檔案，但只想先存 3 個。Staging 讓你主動挑選要存什麼。

You edited 10 files but want to save only 3 — staging lets you pick.

用 Claude Code 時

When using Claude Code

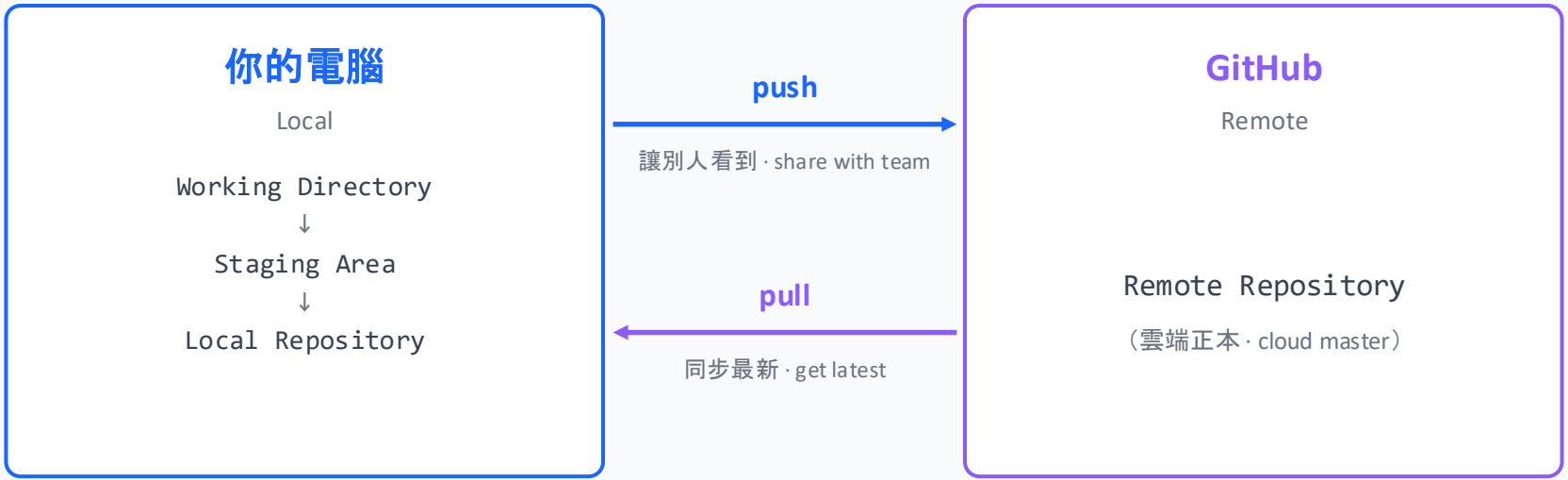
Staging 通常自動處理

Staging is handled automatically — say "save these changes"

ARCHITECTURE

本機 vs 遠端

Local vs Remote — a one-to-one link



Clone 之後，本機資料夾和 remote repo 建立「一對一」連結，之後 push / pull 都透過它同步

After clone, local folder & remote repo are linked 1:1 — push / pull uses that link

FILE STATES

同一份 Header.tsx，三種狀態

Three states of one file — why colleagues can't see your edits

Case A

改了，還沒 commit

Edited, not committed

本機 Local	新版 New
遠端 Remote	舊的 Old

只有你看得到

Only you see it; can be discarded

Case B

commit 了，還沒 push

Committed, not pushed

本機 Local	新版 New
遠端 Remote	舊的 Old

存檔在本機

Saved locally, team still can't see

Case C

push 了

Pushed

本機 Local	新版 New
遠端 Remote	新版 New ✓

團隊可以 pull 到了

Team can now pull your changes

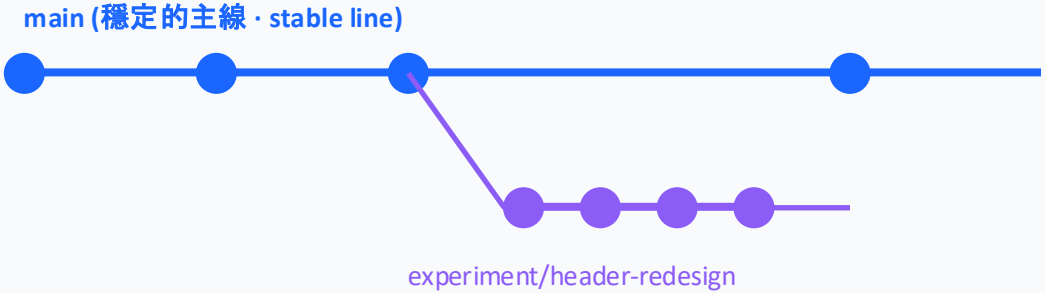
Git 不會自動同步：你 commit 了 ≠ 別人看得到，必須 push 才會更新遠端

Git doesn't auto-sync — commit ≠ team can see it. You must push.

BRANCH

Branch — 平行世界的工作空間

A parallel universe for experiments



為什麼要用 Branch？

Why branch?

- 想試新東西又怕改壞
  Try without risk
- 團隊多人協作
  Team collaboration
- 同時做兩個任務不混一起
  Keep parallel tasks separate

核心價值：Branch 上的改動只有你看得到。失敗就刪，成功再合併。

Core value: changes on a branch are yours alone — delete if it fails, merge if it works.

BRANCH

Branch 命名慣例

name/type/short-description — name first in team repos, lowercase with hyphens

`feature/`	新功能、新頁面 New features / pages	`karen/feature/user-profile`
`fix/`	修復問題 Fix a bug	`kevin/fix/carousel-overflow`
`experiment/`	實驗性質，可能不會合併 Experiments, may not merge	`shuhan/experiment/dark-mode-test`
`refactor/`	重構、整理程式碼 Clean-up / restructuring	`peter/refactor/component-structure`

團隊 repo 把自己名字放最前面，避免和別人撞名、也方便 review 時一眼認出是誰的 branch

In team repos, put your name first — prevents name clashes and makes ownership obvious during review.

BRANCH

何時該用 Branch？

When to branch — a simple rule

✓	想試新東西，不確定會不會用 Trying something uncertain	失敗可直接刪掉 · Delete on failure
✓	會花好幾天的功能 Multi-day feature	避免半成品影響主線 · Protect main from WIP
✓	要交付給別人 review Needs review from others	團隊通常要求 PR · PRs expected
✗	個人 repo 的小 typo Tiny typo in personal repo	直接 main commit 就好 · Just commit to main
✓	公司 / 團隊 repo 的任何改動 Any change in team repos	通常有 branch 保護 · Branch protection

簡單判斷法：個人 repo 小改直接 main，實驗開 branch；團隊 repo 一律 branch

Rule of thumb: personal repo — small edits on main, bigger ones on a branch. Team repo — always branch.

BRANCH

實戰：用 Branch 做 Dark Mode 實驗

In practice — try dark mode on a branch (3 steps)

1 開實驗用 branch

Open an experiment branch

對 Agent 說 · Tell agent

「開一條 branch 叫 experiment/dark-mode」

"Open branch experiment/dark-mode"

你已經在新 branch

You're on a new branch

2 隨意改、隨意 commit

Edit and commit freely

對 Agent 說 · Tell agent

「commit 目前的進度」

"Commit current progress"

進度都存在 branch 上

All progress saved on the branch

3 決定要不要用

Decide — keep or drop

對 Agent 說 · Tell agent

成功 → 「合併到 main」

失敗 → 「刪掉 branch」

Success → "merge to main"

Fail → "delete branch"

main 永遠維持乾淨

main always stays clean

Branch 的價值在於「可以反悔」 · Branches let you undo — main always stays clean

BRANCH

對 Claude Code 說

Talking to Claude Code — natural language → Git commands

你想做的事 What you want	對 Claude Code 說 What to say
開新 branch 開始工作 Start a new branch	「開一條 branch 叫 feature/header-redesign」 *"Open branch feature/header-redesign"*
看目前在哪條 branch Check current branch	「我現在在哪個 branch？」 *"Which branch am I on?"*
切到別的 branch Switch branches	「切到 main」 *"Switch to main"*
做完了，合併回 main Done — merge back	「push 上去，然後開 PR 合併到 main」 *"Push and open a PR to main"*
這條 branch 不要了 Discard the branch	「刪掉 feature/header-redesign 這條 branch」 *"Delete branch feature/header-redesign"*

不用背指令——知道「什麼時候該做什麼事」就夠了 · No need to memorize commands — just know when to do what

PART 2 · PUSH REJECTED

如果 push 被擋了，怎麼辦？

When commit & push gets rejected

```
$ git push
! [rejected]    main -> main (non-fast-forward)
hint: Updates were rejected because the remote contains work
      that you do not have locally.
```

這不是失敗——是 Git 在保護你不要蓋掉別人剛 push 的東西

This isn't failure — it's Git protecting you from overwriting others

> **90% 情況下，對 AI 說「push 被擋了，幫我處理」就解決**
>
> *In 90% of cases, just say "push was rejected, please handle it" — done.*
>
> *細節下面三頁說明· Details on next 3 slides*

SCENARIOS

為什麼會被擋？四種常見情境

Four common reasons — same symptom

最常見

別人剛 push 到同一個 branch

Teammate pushed to the same branch

你在 main 上工作，期間隊友剛推了 commit 上去

常見

多裝置切換沒先 pull

Switched device without pulling first

在 A 機 push 後，B 機繼續工作但忘了 pull

偶發

AI 在背景動過你沒注意

AI did some git work last session

Claude 上一輪 session 自動 commit/push 過，你不知道

罕見

自己的 branch 也被推過

Your own branch was pushed elsewhere

hie-rei 用 <name>/feature/ 命名後幾乎不會撞

症狀都長一樣——你不需要分辨是哪種，AI 看 git 自己就知道

Same symptom — you don't need to diagnose. AI will figure it out.

DESIGNER ACTIONS

對 AI 說：「處理一下」

Three ways to ask AI to handle it

有效的講法（任一即可）

✓ **「push 被擋了，幫我處理」**
"Push was rejected, please handle it"

✓ **「先把別人的更新拉下來再 push」**
"Pull the remote updates first, then push"

✓ **「告訴我為什麼被擋」**
"Tell me why it was rejected"

AI 接著會做的事 · What AI will do

- → 看一下 remote 上是什麼狀態 *Check remote state*
- → 把別人的更新整合進你的 local *Integrate teammate's updates into your local*
- → 再 push 一次 *Push again*
- → 有衝突就問你——不會 silent overwrite *Show conflicts to you, never overwrite silently*

DESIGNER ACTIONS (CONTINUED)

不要說：「強推」、「force」、「蓋過去」

Phrases that trigger force push

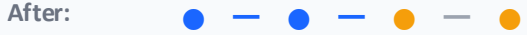
避開這些字眼

- ✗ 強推上去 / force push it
- ✗ 不管它了直接推 / just push it
- ✗ 直接蓋過去 / just overwrite
- ✗ 我要 --force / --force-with-lease

為什麼會出事——Force Push 的後果

(▲ = 隊友剛push 的commit)

AI force push ↓



(▲ 不見了——被你commit 覆蓋)

→ 隊友的工作消失了

Their work is gone

別擔心：hie-rei 的 hooks 已經機械擋下 force push——說錯話也跑不出來。

Don't worry — hie-rei's hooks already block force push, even if your wording is off. (Next slide)

SAFETY NET

專案安全網

You're not defending alone

git hooks by Michael

Michael set up git hooks for this repo

AI 想做剛才提的那些危險操作（force push、git add . 全部、commit 含 secret…）會被自動擋下，不用擔心一不小心犯錯。

Hooks block AI from running the dangerous operations we just talked about — you can't accidentally break things.

⚠ 但是 — 這層保護只在這個 repo 裡有效

This safety net only applies inside this repo

離開這個 repo（你的個人專案、其他 team 的 repo）就沒有這套防呆。

Outside this repo (your own projects, other teams' repos), there's no safety net.

→ 養成不對 AI 說「強推」「force」「直接覆蓋」的習慣，離開 repo 也帶得走。

Build the habit of not saying "force" / "overwrite" — discipline travels with you.

TAKEAWAY

這週記住三件事

Three things to remember from Week 2

01

個人工作也要用 repo 管理

Use a repo for personal work too

不要只存在本機——電腦壞了、檔案誤刪就沒了。建立 personal repo，push 到 GitHub 當備份。

Don't rely on local-only files. Create a personal repo and push to GitHub as backup.

02

在 personal repo 練習 branch

Practice branching in your personal repo

個人 repo 最自由，怎麼玩都不影響別人。熟悉「開 branch → commit → merge」的流程。

Personal repos are the safest playground — master branch → commit → merge before team work.

03

用資料夾結構分層管理

Organize folders in layers

~/work/company/、~/work/personal/，看路徑就知道在哪一層。降低 push 錯 repo 的風險。

~/work/company/ vs ~/work/personal/ — the path alone tells you what layer you're in.

NEXT WEEK

Week 3 / 6

Git 常見名詞解惑

Git Vocabulary — words that look confusingly similar

Commit vs Push

存檔還是分享？· Save locally or share?

Branch vs Fork

同一個 repo 還是複製一份？· Same repo or a copy?

Stash vs Discard

改到一半要切分支怎麼辦？· Mid-edit branch switch

See you next week! · 下週見！